

Report: Chatbot di Dominio

Misica Alessandra
2109775

Monda Elisa
2142650

Prima Parte: Web Parsing

1 Introduzione

La **prima fase** del progetto ha consistito nella realizzazione di una **pipeline end-to-end**, progettata per filtrare il contenuto informativo di pagine web, trasformando il loro codice HTML "sporco" in documenti Markdown lineari e puliti. Questa prima fase di progetto è stata organizzata attraverso i seguenti moduli, pensati per isolare le singole responsabilità.

2 Backend

Contiene il modulo "src" con il file "**server.py**", basato su FastAPI, dove viene implementata la logica degli endpoint richiesti. E' stata aggiunta la funzione "**loading_domains**" per creare la variabile globale "**supported_domains**" con i domini supportati, il dizionario "**PARSER**", con le coppie chiave-valore di dominio e corrispondente parser implementato, e infine la funzione "**complete_gs**" per creare i dizionari contenenti le coppie chiave-valore gold_text - html_text corrispondente e url - html_text locale.

Sono stati utilizzati **modelli Pydantic** per garantire la robustezza dei valori di input e di ritorno di ogni endpoint, rispettando gli schemi richiesti.

3 Parser

Tramite la classe astratta **BaseParser**, viene configurato il WebBrowser in modo che si occupi di gestione della memoria e prevenzione dell' eventuale rischio di essere identificati come "bot" ed essere bloccati da parte dei siti web visitati, tramite l'uso uno "**User-Agent**".

BaseParser delega ad ogni sua sottoclasse (una specifica per ogni dominio) del modulo, la gestione delle relative pagine web. Ogni una di esse contiene quattro variabili, "**targets**",

"**css_exclusions**", "**general_markdown_options**", "**markdown_superfluo**", con le quali definisce le **regole di pulizia** dello specifico dominio, e l'implementazione di un **crawler apposito**. Sono implementati quattro metodi: "**parse_url**", che coordina il recupero del contenuto grezzo seguendo le regole appena citate, "delegando" a "**extract_title**" l'individuazione del titolo, a "**clean_text**" e "**clean_raw_html**" la normalizzazione del testo estratto e l'ulteriore pulizia della stringa HTML grezza.

4 Frontend

Usa **FastAPI** e **Jinja2**, ed è separato dal resto del sistema: riceve dati dal backend, comunicando tramite chiamate **API REST**. L'interfaccia utente, tramite l'implementazione di una pagina HTML, prevede la possibilità di inserimento di un url su cui effettuare il parsing del **testo** o la valutazione dell'**intero dominio di appartenenza**, se supportato. Effettuata la scelta, è possibile visualizzare in due aree di testo la **pagina html**, il testo estratto in formato **markdown**, e due pannelli contenenti, in base alla scelta effettuata, la **valutazione del singolo url** o, tramite il pulsante "**Full GS Eval**", la **valutazione del Gold Standard** di appartenenza.

Nota importante: Si è osservato che l'interazione manuale con l'endpoint di parsing tramite interfaccia HTML potrebbe influenzare i test automatizzati a causa della persistenza della cache del crawler. Il test di un link del dominio di **Amazon.it** non appartenente al GS, costringendo lo scraping online potrebbe generare un output in lingua errata (in inglese), e tale risultato potrebbe essere erroneamente memorizzato e riutilizzato durante la valutazione del Gold Standard, oppure, nonostante l'utilizzo di uno User Agent per simulare il comportamento umano, è possibile che l'estrazione venga bloccata anche se effettuata tramite stringa html locale, portando a un calo fit-

tizio delle metriche di valutazione. **Nonostante l'utilizzo di "UserAgent" e parsing locale, sono stati riscontrati lo stesso dei blocchi come "AN-TIBOT" a seguito di ripetute visite su tutti i domini, specialmente NBA e Amazon, che portano al blocco della visita e dunque parziale estrazione del testo**

5 Evaluation

Il modulo ha la responsabilità di **quantificare** la qualità dell'estrazione e l'efficacia dei parser. Ogni valutazione inizia con una pulizia svolta dalla funzione *"tokenize"*, che tramite espressioni regolari rimuove residui del formato Markdown, restituendo una lista di token puliti. L'accuratezza globale si basa sul confronto statistico tra gli insiemi di token prodotti $P = \text{set}(p_token)$ e attesi $G = \text{set}(g_token)$. I risultati di tale confronto sono espressi da diverse **metriche**, ognuna legata a un aspetto specifico di estrazione del testo:

- **Precision:** $\frac{|P \cap G|}{|P|}$

E' il rapporto tra token corretti trovati rispetto al totale estratto dal parser. Indica la "pulizia" dell'output tramite la percentuale di **parole** estratte che sono **contenuto informativo**.

- **Recall:** $\frac{|P \cap G|}{|G|}$

E' il rapporto tra token corretti rispetto al totale di quelli attesi dall'estrazione. E' la **capacità di "recupero"** del sistema, la percentuale di contenuto originale che il parser è riuscito a salvare.

- **F1-Score:** $\frac{(2 * \text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$

E' la media armonica tra Precision e Recall.

Il range ottimale delle tre metriche è **> 85%**, indice che il parser ha ricostruito fedelmente il testo originale.

Seguono le metriche aggiuntive da noi implementate.

- **Content Density:** $\frac{\text{LenMarkdownEstr}}{\text{LenHTMLSorg}}$

E' il rapporto **Text-to-HTML** tra la stringa finale prodotta dal parser (il Markdown pulito) e l'intera stringa HTML 'grezza' scaricata inizialmente dal sito. Rappresenta quanto "rumore strutturale" è stato rimosso rispetto al testo utile. Il range ideale è **10% - 25%**, indica un **equilibrio ideale tra struttura HTML e testo informativo**.

- **Noise Score:** $\frac{FP}{|P|}$

E' il rapporto i token estratti che non sono presenti nel Gold Standard e il totale della produzione. **Quantifica la presenza di "rumore"** e residui di codice nel testo prodotto, valutando la capacità del sistema di isolare il contenuto utile. Il range ottimale è **< 5%**: meno del 5% del testo estratto è "rumore".

- **Coverage:** $\frac{|P \cap G|}{|G|}$

Anziché concentrarsi sull'accuratezza del parser nel 'ricavare tutti i termini', si focalizza sull'estensione del documento finale, misurandone l'**integrità**. Conferma che l'intero perimetro informativo (titoli, paragrafi chiave) sia stato realmente mantenuto nel risultato finale. Il range ottimale è **> 90%**, indice che nessuna sezione del reale contenuto informativo è stata persa.

Di seguito i risultati totali ottenuti dal sistema, riportati in forma decimale:

Table 1: Risultati completi della validazione per dominio.

Domain	Precision	Recall	F1-Score	Density	Noise	Coverage
Wikipedia	0.990	0.998	0.994	0.118	0.010	0.998
Rotten Tomatoes	0.987	0.999	0.993	0.098	0.013	0.999
NBA	0.996	0.999	0.998	0.029	0.004	0.999
Amazon	0.986	0.928	0.956	0.003	0.014	0.928

6 Peso del Dominio nell'Interpretazione dei Dati

I range ottimali rappresentano punti di riferimento teorici, ma la loro interpretazione deve essere **flessibile**. La validità di un risultato dipende sia dal range che dalla natura del dominio. Il "peso" delle metriche cambia in base alla **difficoltà strutturale del sito**:

- **Wikipedia, Rotten Tomatoes:** In questi domini, i valori rientrano perfettamente nei range ottimali. La Density intorno al 15% e e il Noise Ratio molto basso (circa l'1%), indicano che il parser ha lavorato su un HTML ben strutturato, catturando solo le informazioni utili, mentre Precision, F1-score, Recall e Coverage superiori al 90% indicano una pulizia precisa.
- **NBA:** il dominio ha **Density molto bassa** (2,9%), ben al di sotto del range 10-25%. Ma per un dominio così strutturato è un **risultato**

ottimale: le sue pagine sono “pesanti”, poiché contengono una quantità massiccia di codice non testuale (script per il caricamento di video, tabelle statistiche dinamiche). In questo scenario, il testo leggibile è solo una piccolissima frazione del peso totale del file, dunque la densità è bassissima. Ciò non implica che il parser non abbia lavorato bene, e lo conferma il valore di Noise Ratio, che è 0.4% . Significa che il parser ha saputo ignorare il 97% di codice “spazzatura”, estraendo interamente il contenuto utile.

- **Amazon:** presenta una **Density molto bassa** (0.3%), ma questo dato riflette fedelmente la struttura della pagina, dove il codice HTML (composto da migliaia di tag ‘spazzatura’) sovrasta il testo realmente informativo, perciò è atteso un simile valore di densità. Osservando anche le restanti metriche, si ha conferma che la **pulizia** sia stata accurata e **ottimale** per la struttura di pagina analizzata.

7 Bonus/Extra

- **Adattabilità di "RottenTomatoes"**

L’approccio **adattivo** di Rotten Tomatoes si distingue per la sua capacità di riconoscimento del contesto tramite le differenti tipologie di URL. Il dominio assegnato era *"editorial.rotentomatoes.com"* con focus sulla sezione *"/article"*. Tuttavia, sono state scelte anche alcune pagine con strutture diverse (es. guide, classifiche o news brevi). Abbiamo implementato un **controllo condizionale sull’URL**: se il sistema rileva una sottosezione specifica (diversa da */article*), il parser estende dinamicamente la lista dei target e inserendo selettori ed esclusioni css mirate. Questo permette al parser di mantenere una **copertura maggiore** e una **precisione costante** anche su pagine che si discostano dal template editoriale standard, rendendo lo strumento estremamente versatile e dinamico.

Seconda Parte: Progetto di tesi personalizzato ChatBot Domain

1 Introduzione

La **seconda fase** del progetto costituisce l’evoluzione incrementale della pipeline end-to-end sviluppata nella prima parte. Il sistema di web

parsing è stato integrato e completato all’interno di un’**architettura** documentale e conversazionale di **tipo RAG (Retrieval-Augmented Generation)**. L’**obiettivo** fondamentale di questa nuova sezione è la **realizzazione** di un **Chatbot avanzato**, capace di interpretare e rispondere a domande formulate direttamente dall’utente, operando in completa autonomia attraverso la rete Internet per la ricerca delle fonti, l’estrazione dei testi tramite gli strumenti realizzati nella prima parte, e l’elaborazione dei contenuti per la generazione della risposta utilizzando embedding e chunking.

Il presente progetto rappresenta lo sviluppo di un Chatbot di dominio basato su un’architettura RAG (Retrieval-Augmented Generation), interamente implementato in ambiente Python. Il sistema, progettato includendo un’interfaccia grafica che realizza una chat, accoglie le domande degli utenti, eseguendo per prima cosa una validazione logica basata su regole di dominio prestabilite, risoluzione del soggetto e di riferimenti interni allo storico della conversazione. Successivamente, una volta costruita la query per la ricerca, vengono presi dal web i siti URL più pertinenti, tramite i motori di ricerca DuckDuckGo e Tavily, e ne vengono elaborati i testi, utilizzando i parser specifici di dominio implementati nella prima parte del progetto, a cui è stato aggiunto un parser di fallback per coprire anche siti generici. La gestione e l’indicizzazione delle informazioni estratte si articolano attraverso una fase di chunking — affidata all’algoritmo RecursiveCharacterTextSplitter della libreria LangChain — e una fase di embedding, per la quale è stato integrato il modello intfloat/multilingual-e5-small. I vettori e i dati così strutturati vengono memorizzati all’interno del database MariaDB. Infine, il motore di generazione delle risposte è guidato da un modello di linguaggio Large Language Model (LLM) open-source Qwen 2.5 da 3 miliardi di parametri (3B), prelevato dalla piattaforma Hugging Face. L’esecuzione dell’LLM avviene localmente tramite l’infrastruttura llama.cpp, scelta per ottimizzare al massimo le prestazioni della CPU, sfruttando la quantizzazione del modello.

2 Flusso Logico-Operativo del Sistema

Il funzionamento del sistema si articola in una sequenza coordinata di operazioni logiche:

- **Inoltro della domanda e controllo formale:** Tutto ha inizio nell’**interfaccia grafica** della

chat (*"chat.html"*), dove l'utente inserisce il proprio **quesito**. Il server di frontend (*"chat_frontend.py"*) accetta la richiesta e la **inoltra** al backend (*"server_chatbot.py"*); qui, i dati vengono **strutturati** e verificati attraverso rigidi protocolli di validazione (i modelli Pydantic) per garantirne la **correttezza formale**, per poi essere trasmessi al gestore centrale (*"chat_logic.py"*).

- **Analisi del contesto e sicurezza:** Il gestore centrale interroga lo strato di memorizzazione (*"database.py"*) per recuperare la **cronologia dei messaggi** della sessione attiva. La nuova domanda e lo storico vengono aggiunti nel prompt delle regole da seguire per la validazione della richiesta (*"llm_prompt.py"*) e all'applicazione di controllo del modello locale (*"llm_client.py"*) per una **verifica di sicurezza preventiva**: se la richiesta è **dannosa** o non conforme viene immediatamente rifiutata (*"REJECTED"*). Se invece è **valida**, il sistema risolve i riferimenti interni (come pronomi e soggetti ambigui) sfruttando lo storico dei messaggi di quella conversazione per **reformulare l'input** in una domanda autonoma ed esplicita; **se tuttavia** il modello rileva che **non vi è alcun legame** con lo **storico precedente**, la query viene lasciata **immutata** e rielaborata **singolarmente**. Successivamente, si controlla se l'argomento della richiesta viene coperto da uno dei quattro **domini** noti **supportati** dal sistema (*"domains.json"*). Qualora la domanda **non possa essere soddisfatta da tali domini**, l'LLM valuta **autonomamente** quale potrebbe essere la fonte web **più accurata**. Se la **richiesta è estremamente generica** (es. "che giorno è oggi?"), il dominio viene impostato a **null**. In questa fase, il prompt impone all'LLM di **valutare anche la dinamicità dell'argomento**: se l'informazione è soggetta a variazioni repentine, il flag *"is_dynamic"* viene impostato a **True** per segnalare che la cache locale va ignorata, così da permettere un recupero aggiornato delle fonti necessarie.
- **Controllo della cache e ricerca sul Web:** Prima di navigare in rete, il sistema esegue una **query di controllo** sulla **cache** del database per verificare l'eventuale presenza di materiale già salvato utile a coprire

l'argomento della ricerca corrente **entro un limite di 5 giorni**. Se i dati sono aggiornati, vengono letti direttamente dalla memoria locale risparmiando tempo. **In caso contrario**, si attiva il motore di ricerca configurato (*"DDGSearchEngine.py"* o *"TavilySearchEngine.py"*), che interroga la rete **in tempo reale**, isolando un **massimo di 5 siti web** utili e dando la **priorità assoluta** ai **domini supportati** (o al dominio scelto dalla LLM). Per **massimizzare la copertura**, il modulo esegue una **doppia query**: una **mirata** (utilizzando l'operatore *"site:"* sul dominio scelto dall'LLM, ma solo se quest'ultimo **non è null**, evitando crash di sistema in caso di domande generiche) e una **generica**, utilizzando solo la domanda e senza la specifica del dominio.

- **Estrazione e pulizia del testo:** I link individuati vengono inviati, tramite un **modulo di comunicazione interno** (*"minerva_client.py"*), al server di estrazione (*"server.py"*). Se una pagina appartiene ai **domini noti**, viene analizzata dai **parser specifici**; se appartiene a un **sito esterno**, la richiesta viene indirizzata ad un nuovo parser generico (*"fallback_parser.py"*) che, sfruttando le librerie **Crawl4AI** e **BeautifulSoup** e tag CSS generici, pulisce l'HTML ed estrae il testo utile.
- **Segmentazione e recupero semantico (logica RAG):** Il testo **normalizzato** viene diviso dal **modulo di segmentazione** (*"chunking.py"*) in **piccoli blocchi omogenei** da 800 caratteri. Subentra poi il modulo che svolge l'embedding (*"embedding.py"*), che sfrutta un modello locale (*"multilingual-e5-small"*) per trasformare questi frammenti testuali in **vettori numerici densi**. Calcolando la somiglianza matematica (cosine similarity) tra il vettore della domanda e quelli dei blocchi di testo, il sistema **scarta i contenuti non pertinenti** e **seleziona i 3 frammenti con il valore di correlazione più elevato**.
- **Generazione e risposta in tempo reale:** La domanda iniziale e i 3 frammenti scelti vengono **integrati come contesto nel prompt di istruzioni** generato da *"llm_prompt.py"*, che "costringe" il modello linguistico locale a rispondere basandosi **esclusivamente** sul tale contesto fornito, **azzerando il rischio** di in-

venzioni o **allucinazioni** da parte del modello. A questo punto, il modulo di controllo del modello avvia la **generazione della risposta**. Le parole vengono inviate **in streaming** al frontend, consentendo alla pagina web di mostrarle in tempo reale tramite l'algoritmo di conversione *"marked.js"*, inserendo in coda anche tutti i link usati per l'estrazione di informazioni come **fonti** cliccabili. Al termine del processo, un'operazione asincrona permette al database di salvare la conversazione e di **archiviare i blocchi di testo estratti** con i **rispettivi vettori**.

Questa seconda fase di progetto è stata organizzata attraverso i seguenti moduli e cartelle, pensati per isolare le singole responsabilità.

3 Backend

Il modulo di Backend rappresenta il **core logico, decisionale e di elaborazione dell'intero sistema**. È strutturato come un macro-contenitore che racchiude al suo interno **quattro cartelle principali** collocate sullo stesso livello:

- **evaluation e gs_data**: moduli interamente dedicati alla quantificazione della qualità dell'estrazione e alla gestione del Gold Standard. Poiché il loro funzionamento ed efficacia sono rimasti invariati rispetto alla prima fase (dove sono stati ampiamente trattati), non saranno oggetto di ulteriore approfondimento in questa sezione.
- **parser**: la cartella dedicata alla trasformazione del codice HTML in testo pulito. In questa seconda fase, oltre ai parser specifici per ognuno dei domini supportati, abbiamo implementato un parser generico aggiuntivo, per garantire l'estrazione di informazioni da qualsiasi sito web, per gestire il fallback della scelta di un dominio diverso da quelli noti.
- **src**: all'interno di questo modulo risiede la logica applicativa del chatbot, suddivisa in sotto-cartelle specializzate con responsabilità isolate.

\parser\fallback_parser.py

A differenza dei parser specifici sviluppati nella prima fase del progetto, il **FallBackParser** è stato progettato per essere **universale**, consentendo così la **pulizia di qualsiasi pagina web** grazie a due precise scelte fatte nel codice:

- In primo luogo, all'interno delle variabili *"FALLBACK_EXCLUSIONS"* e *"FALLBACK_GEN_EXCLUDED_TAGS"* sono stati inseriti tutti i **tag e le classi CSS più ricorrenti nel web** contenenti banner pubblicitari e popup dei cookie, permettendo a BeautifulSoup di effettuare una **pulizia standardizzata** efficace su **qualunque** sito.
- In secondo luogo, all'interno del dizionario *"FALLBACK_MARKDOWN_OPTIONS"* è stato forzato a **True** il flag *"ignore_links"* insieme a quello delle immagini. Questo ha permesso di ottenere un Markdown più pulito, per evitare di **"inquinare"** i vettori di embedding, ottimizzando il successivo calcolo della similarità del coseno all'interno del modulo responsabile.

\src\api_server

Questo modulo espone le interfacce di comunicazione di rete del backend verso l'esterno. Al suo interno troviamo i seguenti file:

- **server_chatbot.py**: È l'**interfaccia di comunicazione** del sistema. Attraverso la variabile di configurazione *"CHOSEN_ENGINE"*, il sistema applica una **logica condizionale** per determinare **quale motore di ricerca utilizzare**: se impostata su **DuckDuckGo**, viene caricato il relativo modulo di ricerca che offre un servizio di consultazione gratuito e illimitato. In alternativa, abbiamo permesso l'attivazione di un ulteriore motore di ricerca, **Tavily**, ottimizzato per contesti LLM ma vincolato all'uso di token. Il provider scelto viene poi iniettato direttamente all'interno di *"ChatLogic"*, garantendo la separazione dei vari componenti, mentre la solidità dei dati in ingresso viene garantita dai modelli di validazione **Pydantic**.

Sono presenti diversi **endpoint**; il principale è *"chat"*, che gestisce il **flusso conversazionale** restituendo un oggetto **StreamingResponse**. Questa risposta implementa il protocollo **Server-Sent Events (SSE)** per trasmettere i token all'utente **in tempo reale** man mano che vengono generati dall'LLM. Ci sono anche una serie di **endpoint secondari** per il database, dedicati rispettivamente alla **gestione delle sessioni** (*"chat_new_session"*, *"chat_all_session"*, *"chat_messages"*) e alle

operazioni di **pulizia e manutenzione** della memoria ("*/erase_history*", "*/erase_session*", "*/erase_all_sessions*").

- **server.py**: Mantiene la medesima **struttura FastAPI** ereditata dalla prima fase del progetto (gestione dei domini supportati, consultazione del Gold Standard e calcolo delle metriche), con un'unica e fondamentale aggiunta architetturale: l'endpoint "*/generic_parse*". Questa nuova rotta espone pubblicamente l'istanza globale "*GENERIC_PARSER*", consentendo di ricevere ed estrarre informazioni da **qualsiasi URL di un dominio non presente tra quelli supportati**. In questo modo si fornisce una **via di recupero universale** senza intaccare i vincoli e i filtri stringenti del vecchio endpoint "*/parse*".

\src\chat_logic

Come descritto sopra, questo modulo funge da gestore centrale dell'intera pipeline RAG. La classe coordina i vari moduli di sistema attraverso una struttura suddivisa in metodi chiave:

- **__init__**: Inizializza il **modulo di memoria vettoriale** ("*self.vector_store*") come un **Singleton** condiviso e protetto da meccanismi di **mutua esclusione** ("*threading.Lock*"). All'inizio di ogni esecuzione viene invocato il metodo "*.clear()*" per **azzerare l'indice volatile** ed **eliminare qualsiasi rischio di contaminazione** dei dati tra utenti diversi.
- **check_db_relevance**: Gestisce l'interrogazione dello **strato di persistenza** e la **validazione** della **cache locale**. Per ottimizzare le prestazioni, converte i blocchi informativi in **matrici NumPy** per calcolare la **cosine similarity**. Calcola la media dei due migliori *chunk* per documento e applica i **filtri di soglia di precisione** ("*threshold=0.82*") e di **limite temporale** ("*REPARSE_THRESHOLD_DAYS = 5*") menzionati precedentemente, determinando **se sia necessaria o meno una nuova estrazione web**.
- **execute_chat_flow**: È il **punto di ingresso principale** che coordina l'effettiva **esecuzione asincrona** della pipeline. Sfrutta il costrutto `asyncio.as_completed(tasks)` per **parallelizzare** il *parsing* delle **risorse web**. Coordina infine la **generazione dei token**

dell'LLM in **tempo reale** tramite la funzione "*sse_format*" e, a trasmissione conclusa, assicura il **salvataggio della cronologia** dei messaggi e delle fonti usate sul database.

\src\client

Lo scopo principale di questo modulo è **astrarre le chiamate di rete**, offrendo al resto dell'applicazione metodi semplici e pronti all'uso per **dialogare con le API**.

- **minerva_client.py**: Gestisce la **comunicazione HTTP** tra la **chat** e il **server che contiene i parser per l'estrazione dei test**, con lo scopo di **inviare** gli URL, **richiederne** la pulizia strutturale ed **evitare** blocchi del sistema, restituendo **None** in caso di **errore** o timeout di rete. Il funzionamento si basa sull'**interazione di tre elementi principali**. Il **dizionario** "*SUPPORTED_DOMAINS*" mappa i quattro domini predefiniti del progetto alla rotta "*parse*". Quando viene invocata la funzione "*parse_url*", il sistema **ricava l'host** dell'URL tramite "*url_parse(url).hostname*" e interroga questo dizionario sfruttando il metodo "*.get()*"; qualora l'host **non** venga trovato tra quelli supportati, viene selezionata **automaticamente** la **stringa di fallback** "*generic_parse*". Questo valore permette di **comporre l'indirizzo finale** dell'API verso cui effettuare la chiamata **HTTP GET**, estraendo poi dal JSON di risposta il **testo pulito** mediante il campo "*.get('parsed_text')*". Infine, il metodo "*get_parsed_text_from_urls*" **centralizza** l'elaborazione dei collegamenti ricevuti dai motori di ricerca, **scorrendo la lista degli URL** in input per sottoporli singolarmente a "*parse_url*" e **inserendo i risultati validi** in una struttura "*list[tuple[str, str]]*", che **mappa ogni indirizzo web** al rispettivo **testo Markdown** normalizzato.

\src\mariadb_data

Questo modulo costituisce lo strato di persistenza dell'applicazione, con il compito di conservare la cronologia delle conversazioni e archiviare i documenti estratti dal web.

- **database.py**: **definisce l'infrastruttura di persistenza** dell'applicazione, ovvero il database, utilizzando **MariaDB**, e implementando un **meccanismo di Connection Pooling** globale mediante la funzione

`"_get_pool()"`. Tale componente è configurato come un **Singleton** a **inizializzazione pigra** con una **capacità massima di 10 connessioni simultanee**, ottimizzando l'allocazione delle risorse di rete. Per **monitorare i thread attivi ed evitare leak di risorse**, la funzione `"get_connection()"` sovrascrive il metodo nativo di chiusura tramite la `"closure monitored_close()"`, la quale **decrementa il contatore globale** delle connessioni in uso concorrente **ogni volta** che una **risorsa viene rilasciata** e riposta nel pool. L'**architettura relazionale** del database viene inizializzata nella funzione `"init_db()"` attraverso **quattro tabelle principali** legate da **vincoli di integrità referenziale** con opzione `"ON DELETE CASCADE"`. La tabella ***documents*** funge da **cache centralizzata** per i testi normalizzati estratti dal web, mentre ***document_chunks*** ne **memorizza** i rispettivi **frammenti testuali accoppiati ai vettori di embedding**, serializzati in formato testuale tramite l'istruzione `"json.dumps()"`. La **gestione dello storico conversazionale** è invece affidata alle tabelle speculari ***chat_sessions*** e ***chat_messages***, strutturate per **memorizzare le sessioni utente**, i **ruoli dei messaggi** e i **riferimenti alle fonti informative** utilizzate dall'LLM. Per quanto riguarda le **operazioni di lettura e scrittura**, il sistema impiega **query parametriche native** per prevenire vulnerabilità di tipo **SQL Injection**. Il salvataggio dei dati in `"save_document_with_chunks()"` adotta il costrutto `"INSERT IGNORE"` per **prevenire collisioni su URL duplicati** e **ripulisce preventivamente** i vecchi frammenti tramite un'istruzione di `"DELETE"`, **garantendo la coerenza** dei vettori associati. In fase di recupero, i metodi `"get_all_chunks_with_urls()"` e `"get_multiple_cached_embeddings()"` **massimizzano le prestazioni della pipeline RAG deserializzando** le stringhe JSON direttamente in **array NumPy float32**. Infine, la **manutenzione automatica** della cache viene eseguita dalla funzione `"erase_history()"`, la quale **invalida i dati obsoleti eliminando i record** salvati da oltre **7 giorni** o appartenenti a domini le cui informazioni sono molto **variabili** (come nel caso di `www.nba.com`, le cui informazioni vengono aggiornate spesso).

`\src\llm_data`

Questo modulo si occupa di strutturare le istruzioni e i contesti informativi da inviare al modello linguistico locale, traducendo la richiesta dell'utente e i frammenti RAG recuperati in un prompt ottimizzato.

- **llm_client.py**: La classe **LLMClient** gestisce l'inferenza del modello locale tramite **llama_cpp**. La funzione `"get_llm_client()"` ne **garantisce l'istanza unica, evitando caricamenti duplicati** in memoria da parte di thread concorrenti usando un **semaforo globale**. Poiché l'esecuzione si appoggia **interamente sulla CPU**, i **metodi sono protetti dal blocco atomico** `"self.cpu_lock"` per **evitare collisioni**. Il client svolge **due compiti principali nella pipeline**. Il **primo** è la **validazione della query utente** tramite il metodo `"generate_validation()"`, che opera con una **temperatura rigida (0.1)** e una **penalità di ripetizione (1.2)** per **forzare il modello a restituire un output** strutturato in formato **JSON**; un **blocco try/except** intercetta **eventuali anomalie strutturali** generando un JSON di emergenza per non bloccare il sistema. Il **secondo compito** è la **generazione della risposta finale unendo la richiesta al contesto RAG**, implementata in modalità **classica** tramite `"generate()"` e in **streaming** tramite `"stream()"`. Quest'ultimo metodo attiva la **generazione progressiva** e estrae e restituisce i token in **tempo reale**.
- **llm_prompt.py**: La classe **PromptManager** **centralizza e isola** la logica dei prompt dell'applicazione attraverso **metodi statici** (`@staticmethod`), strutturando le direttive fornite al modello linguistico per le **due fasi distinte** della pipeline: la **validazione dell'intento** e la **generazione della risposta basata sul contesto**. Il **primo blocco funzionale** si occupa della **formattazione dell'output** tramite `"get_validator_role()"`, che **istruisce rigorosamente l'LLM a escludere** qualsiasi testo conversazionale prima o dopo le parentesi graffe, forzando la generazione del solo formato JSON. A questa direttiva si aggancia il metodo `"get_intent_from_user_query()"`, concepito per l'**analisi** e la **strutturazione della richiesta**. La funzione prende in **input la query**

utente corrente e lo storico della conversazione, estraendo **gli ultimi quattro messaggi** (`"chat_history.strip().split('/n')[-4:]"`) per **mantenere una finestra di memoria rilevante** senza sovraccaricare il contesto. Il **prompt** strutturato **risultante** impone **regole stringenti** per la **risoluzione** dei pronomi, soggetti impliciti e riferimenti a messaggi precedenti (generando la chiave `"expanded_query"`), **disattivando** l'eventuale riformulazione della query, lasciandola **inalterata** se l'LLM determina che **non** c'è alcun legame con il contesto precedente. Gestisce inoltre la **classificazione della richiesta** (tramite i flag `"REJECTED | VAGUE | CLEAR"`), la **mappatura del dominio di ricerca web** e l'**analisi della dinamicità dei dati** (`"is_dynamic"`). A livello di dominio, il prompt istruisce il modello affinché, se **non** viene scelto nessuno dei quattro domini predefiniti come idoneo al recupero di informazioni sul web, viene valutato dal modello stesso quale potrebbe essere quello **più accurato** tra quelli sul web, e lo inserisce nel JSON; se non riesce a prevederne uno o se la domanda è del tutto **generica** e **atemporale**, assegna esplicitamente il valore **null**. Il flag `"is_dynamic"` serve invece a indicare se l'argomento **cambia** particolarmente **spesso**, salvando questo valore affinché venga poi letto da **chat_logic** per decidere se procedere con un **nuovo parsing online** (nel caso in cui si stesse riutilizzando quello stesso sito dal database per l'estrazione di informazioni). Il tutto viene fatto includendo una serie di **esempi** per **standardizzare il comportamento** del modello in base ai diversi **scenari d'uso**. Il **secondo blocco** operativo è rappresentato dal metodo `"get_rag_prompt()"`, il quale **definisce le regole di generazione della risposta finale** da mostrare all'utente. Questo prompt configura l'LLM come un puro assistente di riscrittura **privo di conoscenza interna**, imponendo il vincolo di utilizzare **esclusivamente** le informazioni presenti nel parametro `"context"`. Le regole integrate **forzano** il modello a **dichiarare** esplicitamente l'**assenza di dati** qualora la cache o la ricerca web non abbiano prodotto risultati utili, a **mantenere una sintassi naturale**, e ad usare la stessa lingua rilevata nella richiesta dell'utente.

`\src\rag`

Questo modulo si occupa di dividere i documenti in blocchi ottimizzati, convertirli in vettori numerici e selezionare solo i frammenti più rilevanti per generare una risposta precisa e priva di allucinazioni.

- **chunking.py:** La classe **TextChunker** si occupa della **scomposizione** dei **testi** normalizzati in **blocchi di minori dimensioni** per **ottimizzare** la fase di **indicizzazione vettoriale** all'interno della pipeline RAG. L'architettura del modulo si basa sul componente *RecursiveCharacterTextSplitter* fornito dalla libreria **LangChain**, configurato nel costruttore per generare **frammenti** con una **dimensione massima di 800 caratteri** (`"chunk_size=800"`) e una **sovrapposizione di 100 caratteri** (`"chunk_overlap=100"`) per **preservare la continuità semantica** tra blocchi **contigui**. Al fine di mantenere l'**integrità logica delle informazioni**, l'algoritmo **non** effettua **tagli arbitrari** ma **valuta** in ordine **gerarchico** una serie di **separatori** testuali, partendo dai doppi a capo fino ai singoli spazi tipografici. Infine, il metodo `"split()"` **riceve** la stringa di testo originaria, **verifica** tramite un controllo condizionale l'eventuale assenza di contenuto per evitare computazioni ridondanti e **restituisce** la lista finale dei frammenti elaborati pronti per il modulo di embedding.
- **embedding.py:** La classe **EmbeddingModule** gestisce un **database vettoriale in memoria volatile** basato su *SentenceTransformer*. La funzione `"get_embedding_module()"` ne assicura l'**istanza unica**, mentre il metodo `"clear()"` **azzerà le liste interne "vectors" e "chunks"** sotto la protezione del lucchetto locale `"self._lock"` **all'inizio di ogni chat** per **evitare contaminazioni** tra più utenti connessi in parallelo. Il flusso dei dati si articola in **tre fasi principali**: La codifica viene eseguita dal metodo `"encode()"`, che **assegna i prefissi del modello E5** aggiungendo `"query: "` alle **domande** e `"passage: "` ai **testi**. La funzione `"add_documents()"` esegue il **calcolo intensivo degli embedding** all'esterno del lock per non bloccare il sistema, per poi **inserire** in modo **sicuro** i vettori e i frammenti di testo associati nelle liste della classe tramite un blocco `"with self._lock"`. Per i **dati già presenti** nel

database, il metodo `"add_to_index()"` **salta la codifica** inserendo **direttamente** i record in memoria. La ricerca semantica sfrutta la velocità dei calcoli matriciali di NumPy. La funzione `"_calculate_cosine_similarity"` **confronta l'intera matrice dei vettori** dei chunk di testo già salvati nel database con il vettore della **query corrente** in un'unica operazione, **gestendo** in modo sicuro i **denominatori a zero** tramite `"np.divide"`. Infine, il metodo `"search()"` **calcola le similarità**, **filtra** i risultati con `"np.where"` in base alla soglia `"threshold=0.80"` e **ordina** gli indici in modo **decrescente**, **restituendo** alla chat i **primi 3 frammenti di testo più rilevanti**.

`\src\web_search_engine`

Il modulo di ricerca si basa su una **struttura espandibile** che separa la **logica del sistema** dall'**interazione reale con i motori di ricerca esterni**. La classe astratta `BaseSearcher` (contenuta nel file `"base_searcher.py"`) **stabilisce le regole comuni** per tutti i **motori di ricerca**: essa **impone** l'implementazione obbligatoria del metodo `"get_search_data()"` e **inizializza** una lista di domini autorizzati (`"whitelisted_domains"`) che serve a filtrare i risultati e tenere solo quelli importanti.

- **ddg_engine.py**: La classe `DuckDuckGoSearcher` estende la logica introducendo una **lista locale (blacklist)** di **domini da escludere**, come **piattaforme multimediali e social**. Quando viene invocato il metodo `"get_search_data()"`, il modulo esegue una **doppia interrogazione concorrente** tramite la libreria **DDGS**: una **ricerca mirata vincolata al dominio scelto dall'LLM** (utilizzando l'operatore `site:`) e una **query generica libera**. Questa **doppia interrogazione** permette di raccogliere il **maggior numero di link utili per costruire un contesto ricco**. Tuttavia, per evitare errori, la **query condizionale con operatore "site:"** viene lanciata **solo se il dominio estratto dall'LLM non è null** (caso tipico delle domande generiche come "che ore sono"), usando altrimenti la sola ricerca "libera". I risultati grezzi vengono **uniti, ripuliti dai duplicati ed eliminati** se appartenenti ai domini **bloccati**. Infine, un **filtro gerarchico** riordina l'output posizionando in testa i link che corrispondono alla `whitelist`, cioè la lista dei domini supportati dal nostro sistema.

- **tavily_search_engine.py**: La classe `TavilySearchEngine` si appoggia alle API dedicate di Tavily tramite il `TavilyClient`, e segue la stessa logica descritta sopra per il motore di ricerca DuckDuckGo.

4 Frontend

Il modulo di Frontend costituisce lo **strato di presentazione del sistema**, con il compito di **servire l'interfaccia grafica all'utente** e **gestire la comunicazione in tempo reale** con il **backend**. La sua struttura si sviluppa in modo molto lineare attraverso **due cartelle principali**:

- **src**: Qui troviamo sia il frontend svolto per la prima parte del progetto, sia il file **chat_frontend.py**, ovvero lo script attualmente in uso che si occupa di **gestire la connessione asincrona e lo streaming dei messaggi**.
- **templates**: Questa cartella contiene le interfacce grafiche visualizzate dal browser del client. Al suo interno è presente il file **chat.html**, che rappresenta il template attualmente utilizzato per realizzare il **chatbot**, e il file **index.html**, utilizzato nella versione precedente del progetto (ora inutilizzato).

Approfondiremo solo l'analisi del nuovo frontend, in quanto quello precedentemente implementato è stato già ampiamente commentato nella prima parte del report.

`\src\chat_frontend.py`

Questo file gestisce lo **smistamento dell'interfaccia utente**, facendo da **intermediario** tra il **browser** e il **vero backend del sistema**. Per prima cosa **configura il client di comunicazione globale "http_client"** **eliminando qualsiasi limite di tempo** per la lettura con `"read=None"`, una scelta indispensabile per non far morire la connessione mentre l'LLM sta ancora elaborando la risposta.

La prima funzione che incontriamo è `"get_chat_page"`, che **si attiva quando l'utente entra nel sito** e si limita a servirgli la **schermata grafica della chat** caricando il template **chat.html**. Il punto fondamentale è però la funzione `"proxy_chat"`: questa **riceve la domanda** che l'utente ha digitato, la **spedisce** al backend e, anziché aspettare che la risposta sia completa, **usa un generatore asincrono chiamato**

"stream_generator" per **intercettare** i **singoli frammenti di testo** non appena l'LLM li produce. Grazie a *"StreamingResponse"*, questi pezzetti vengono scaricati sullo schermo in tempo reale, parola per parola.

Infine, la funzione *"proxy_database_requests"* si occupa di tutto il resto: **recupera** la **cronologia** dei messaggi, **intercetta** le **richieste** del browser, **ricostruisce** l'URL inserendo i dati della sessione e, una volta ottenuto il via libera dal backend, **restituisce** i dati puliti sotto forma di **JSONResponse**.

\templates\chat.html

Il file **chat.html** definisce semplicemente la **struttura e la grafica dell'interfaccia utente**, stilizzata con la libreria **Tailwind CSS**. All'interno di questo file sono stati implementati alcuni elementi particolarmente interessanti che ottimizzano l'esperienza utente.

Il primo aspetto rilevante riguarda la gestione dei Server-Sent Events (SSE). Nella funzione *"sendMessage"*, il codice non aspetta una risposta HTTP classica, ma **legge i dati dal server un pezzetto alla volta** tramite un **Reader** asincrono. Questo meccanismo permette di distinguere i vari step della pipeline, mostrando all'utente cosa sta succedendo nelle varie fasi di elaborazione e generazione della risposta: ad esempio, se viene ricevuto uno *"status"* il sistema mostra l'animazione di caricamento dei motori di ricerca, se riceve un *"token"* stampa la parola direttamente a schermo, mentre se riceve le *"sources"* si occupa di renderizzare i link cliccabili delle fonti.

Il secondo elemento di rilievo è l'**"effetto macchina da scrivere" controllato**. Per evitare che lo schermo mostri di colpo un blocco unico di testo quando l'LLM conclude la risposta, viene introdotto un buffer denominato *"textQueue"* insieme a un **intervallo temporale fisso** tramite *"setInterval"* impostato a **15ms**. Questo espediente mostra i caratteri in modo **fluid** e **leggibile** per l'utente, parola per parola, convertendo contemporaneamente il testo da Markdown a HTML attraverso l'ausilio della libreria *marked*.

Il terzo aspetto interessante riguarda il **supporto alle sessioni multiple di conversazione**. Tramite una **barra laterale dinamica** inserita nel layout, l'utente ha la possibilità di **creare e mantenere più conversazioni parallele**, potendo saltare da una chat all'altra **in qualsiasi momento**; il sistema si occupa di **svuotare la finestra principale** e **ricaricare la cronologia dei messaggi**

che compongono la sessione appena selezionata, inviando una **richiesta asincrona al backend**. Inoltre, per facilitare l'organizzazione visiva delle chat, l'interfaccia include una funzionalità di **editing rapido** che permette all'utente di **modificare** direttamente a schermo il **titolo di ogni singola conversazione** (sovrascrivendo quello generato di default, che assume il valore della prima richiesta fatta dall'utente in chat), inviando la modifica al database tramite una chiamata fetch in background. Infine, tramite una cella in alto a destra, è possibile cambiare l'utente attualmente connesso alla chat, permettendo la creazione di profili diversi e il salvataggio di conversazioni appartenenti ad essi.